



Mock server

Rationale

Test cases can range in complexity; from testing the return value of a simple helper function, to rendering an entire webclient to simulate interactions across several components/services.

In the latter cases, many interactions will trigger server requests, without which the components or features will stop functioning properly. However, it is important that these requests *do not* land on the actual server, as they could affect the database, which is definitely not something a test should be doing.

To overcome this, each request should be intercepted and replaced by a function emulating actual server responses with test (i.e. fake) data.

Since some of these requests are very common (e.g. ORM calls, such as `web_search_read` or `web_save`, or other methods such as `get_views`), a mock server has been implemented by default for every test that spawns an `env` [\[1\]](#).

These mock servers act independently for each test, can be configured separately, and provide out of the box helpers for the most used routes in Odoo.

[\[1\]](#) A mock server is needed as soon as an environment is spawned because some [services](#) do send server requests as soon as they start.

Overview

A mock server is actually quite simple in itself: it is an object containing a *collection* of all defined mock models, and a *mapping* between routes and callbacks returning the test data.

The mock models themselves hold most of the CRUD logic, as well as the data used to simulate server records.

Once a mock server starts, it hijacks *all* server requests, and for each of them it will check in its *mapping* whether one of its registered routes matches the requested URL. The most notable example of its pre-defined routes is `/web/dataset/call_kw`, which is responsible for calling an ORM method on the appropriate mock model.

be found in the `@web/.../tests/web_test_helpers` module.

Configuration

By default, a mock server is *“empty”*, meaning that it has no defined mock model.

This does not mean that it is useless though, as it will already handle a few pre-defined routes, such as the ones responsible for fetching `menus` and `translations`, which are spawned by **services** as soon as an `env` is spawned.

But this means that ORM methods will fail, as the model that they target has not been defined yet.

To create and define a mock model, you need 2 things:

- a class extending the `models.Model` class;
 - special keys prefixed with a `_` act as metadata holders, like in Python (e.g. `_name`, `_order`, `_description`, etc.) [\[2\]](#) [\[3\]](#);
 - `_records` holds the list of objects representing fake record data;
 - `_views` can be a mapping of view types and XML arches;
 - other **public class fields** [↗](#) will be interpreted as fields (by calling the appropriate method from `fields`);
 - model-specific methods (such as `has_group` for `"res.users"`) can also be defined here.
- calling `defineModels` with the class defined above.

[2] Only a subset of these special keys will have an actual effect. For example, `_inherit` will not work as intended, prefer standard class extension.

[3] These can be altered by each test without thinking about cleaning up: any change performed on a special key will be reverted at the end of a test.

Here is a basic example of a simple, fake, `"res.partner"` model:

```
import { defineModels, fields, models } from "@web/.../tests/web_test_helpers";

class ResPartner extends models.Model {
  _name = "res.partner";
```

```

        { name: "Mitchel Admin" },
    ];

    _views = {
        form: /* xml */`
            <form>
                <field name="name" />
            </form>
        `,
        list: /* xml */`
            <list>
                <field name="display_name" />
            </list>
        `,
    };
}

defineModels({ ResPartner });

```

This code will make these data available for *all* tests in the current test file. Of course, defining a class and calling `defineModels` can also be done from *within* a given test to limit the scope of that model to the current test.

Other methods such as `defineMenus`, `defineActions` or `defineParams` can also be used to configure the current mock server. Most of their API is quite straightforward (i.e. they receive JSON-like descriptions of menus, actions, etc.).

Mock models: requests

Many test cases only require one or a few mock models to work. But sometimes, it is either too bothersome to implement the mocking logic within a model, or a *route* (i.e. server request URL) is simply not associated to a Python model at all.

In such cases, the `onRpc` method is to be called, to associate a route or an ORM method to a callback.

Note

Multiple `onRpc` calls can be associated to the same route / ORM method; in which case they will be called sequentially from last to first defined. Returning a *non-null-or-*

It can be used in 4 different ways:

onRpc : with a route ("/")

When the first argument is a `string` starting with a `"/"`, the callback is expected to be a *route* callback, receiving a `Request_` object:

```
onRpc("/route/to/test", async (request) => {  
    const { ids } = await request.json();  
    expect.step(ids);  
    return {};  
});
```

By default, the return value of these callbacks are wrapped within the `body` of a mock `Response_` object.

This is fine for most use-cases, but sometimes the callback needs to respond with a `Response_` object with custom `status` or `headers`.

In such cases, an *optional* dictionary can be passed as a 3rd argument to specify whether the callback is to be considered *"pure"*, meaning that its return value should be returned as-is to the server caller:

```
onRpc(  
    "/not/found",  
    () => new Response("{} ", { status: 404 }),  
    { pure: true }  
);
```

Note

Using *"pure"* request callbacks can also be used to return anything else than a `Response_` object, in which case the returned value will still be wrapped in the body of a mock `Response_` to comply with the `fetch_` / `XMLHttpRequest_` APIs.

onRpc : with method name(s)

The callback will receive an object containing:

- the *spread* params value contained in the request body (typically: `args` , `kwargs` , `model` and `method`);
- a `parent()` function, which when invoked will call the defined ORM callback *preceding* this one;
- a `route` key, containing the `pathname` of the request (typically: `/web/dataset/call_kw`);
- the `request` object.

```
onRpc("web_read", async ({ args, parent }) => {
  const result = parent();
  expect.step(args[0]); // Contains the list of IDs
  result.some_meta_data = { foo: "bar" };
  return result;
});
```

onRpc : with model name(s) AND method name(s)

When:

- the first argument is a `string` *NOT* starting with a `"/` or a list of `strings` ;
- the second argument is also a `string` or a list of `strings` ;

Then the callback is expected to be an ORM callback, only called when the request's `method` *AND* `model` match the ones given in the arguments.

This works just the same as the above shape, with an added `model` filter:

```
onRpc("web_read", "res.partner", ({ args }) => {
  expect.step(args[0]);
});
```

onRpc : for *every* ORM method/model

When the *only* argument is a callback, it is expected to be an ORM callback to be called for *every* ORM call:

Mock models: fields

Model fields can be declared in 2 ways:

- as **public class fields** [↗](#);
- under the `_fields` special key. For example:

```
test("test view with date fields", async () => {  
    // `_fields` can be assigned over, or extended directly.  
    ResPartner._fields.date = fields.Date({ string: "Registration date" });  
});
```

Field constructors can take a parameters dictionary to dictate their behaviour. It will be required for some of them, like relational fields, which need a `relation` property to work correctly.

There are limits to what can be done with a mock field compared to an actual Python server field, but expect the most basic properties to be supported: `readonly`, `required`, `string`, etc.

`compute` and `related` do work for the most basic use-cases, but do not expect them to function reliably as they would on the actual server.

Note

There are 4 default fields pre-defined for each created model: `id`, `display_name`, `created_at` and `updated_at`. They match their server-side counterpart in their behaviour (e.g. `id` is incremental and `display_name` has a `compute` function similar to its server counterpart), and can be overridden if needed.

Mock models: records

Model records are generated based on each object contained in the `_records` special key *when the model is loaded*. They are validated based on the fields available on the current models; if a property does not match a field defined on the model, an error is thrown.

has started. This key is only used to generate initial records. If records should be added *after* model creation, do it either from the available components in the UI, or through direct ORM calls on the mock server instance.

Mock models: views

Since actual views need an `"ir.ui.view"` model to be declared, mock models use a simplified *mapping* to provide view arches.

The `_view` special key is a dictionary, with its *keys* being view types, optionally accompanied by a view ID, and its *values* being the XML arch string representation.

By default, view IDs are `false`, but can be specified explicitly with a comma-separated key combining the view type and its ID:

```
// Will simulate a list view with no ID (false).
ResPartner._views.list = /* xml */ `
    <list>
        <field name="display_name" />
    </list>
`;

// Will simulate a form view with ID 418.
ResPartner._views["form,418"] = /* xml */ `
    <form>
        <field name="name" />
        <field name="date" />
    </form>
`;
```

Spawning a mock server

Just like in most cases, only one server can be active for a given test.

As mentioned above, creating an `env` will automatically deploy a mock server.

This means that all of these methods will *also* create a mock server, since they do create an `env` :

- `makeMockEnv;`

environment. For that purpose, a `makeMockServer` helper can be called separately to initiate a mock server.

Note

`makeMockServer` should *only* be used by low-level features, such as testing the `rpc` function without the environment. It is not meant to be used as a means to retrieve the current mock server instance. For that purpose, refer to **`MockServer.current`**.

Note

It is to be noted that subsequent calls to `makeMockServer` after a mock server has been started are simply ignored.

Interacting with the server

While most of the server interactions are expected to be done directly or indirectly by production code spawned in the test case, it is sometimes meaningful to bypass the UI and call the mock server directly (e.g. to simulate that another user, somewhere else, somehow, has altered the database).

This can be done by retrieving the `MockServer.current` static property containing the current mock server instance (only after initialization):

```
// Most common ORM methods are provided out of the box by server models,  
// and are synchronous. Although, be careful that this will NOT trigger a  
// UI re-render, and will ONLY affect the (fake) database.  
const ids = MockServer.env["res.partner"].create([  
  { name: "foo" },  
  { name: "bar" },  
]);
```

Tip

`MockServer.env` is just a shortcut to `MockServer.current.env`.



Get Help

Contact Support

Ask the Odoo Community

